

IF2224 FORMAL LANGUAGE AND AUTOMATA THEORY

Arion Compiler - Milestone 2: Syntax Analysis

May 3, 2026

Made Branenda Jordhy - 13524026

Muhammad Nur Majiid - 13524028

Jason Edward Salim - 13524034

Athilla Zaidan Zidna Fann - 13524068

**School of Electrical Engineering and Informatics
Institut Teknologi Bandung
Semester II 2025/2026**

Contents

1	Theoretical Background	3
1.1	Syntax Analysis	3
1.2	Context-Free Grammar (CFG)	3
1.3	Parse Tree dan AST	3
1.4	Recursive Descent Parsing	3
2	Design and Implementation	4
2.1	Overview	4
2.2	Hard-coded Grammar	4
2.3	Recursive Descent Architecture	5
2.4	Parsing Expression: Operator Precedence	6
2.5	Disambiguasi: Assignment vs Procedure Call	6
2.6	Error Handling	7
2.7	Parse Tree Output	7
2.8	Tokenization Flow	7
2.9	Contoh Parse Tree	8
3	Testing	8
3.1	Lokasi File Pengujian	8
3.2	Ringkasan Kasus Uji	8
3.3	Hasil Uji Berdasarkan Screenshot	9
4	Conclusion	11
4.1	Saran	11
	Appendix	13

1 Theoretical Background

1.1 Syntax Analysis

Syntax analysis adalah fase kedua dalam pipeline kompilasi, yang dijalankan setelah *lexical analysis*. Komponen yang melaksanakan tahap ini disebut *parser*. Apabila lexer memproduksi barisan token sebagai representasi leksikal program, maka parser bertugas memverifikasi bahwa urutan token tersebut membentuk *kalimat* yang sah dalam bahasa, yaitu struktur yang dapat diturunkan dari *grammar* bahasa.

Selain melakukan verifikasi, parser juga membangun representasi hierarkis yang disebut *parse tree*. Representasi ini menjadi masukan bagi tahap analisis semantik dan code generation pada milestone berikutnya. Tujuan parser dapat diringkas sebagai berikut.

- Memastikan urutan token membentuk struktur sintaks yang valid menurut grammar.
- Mendeteksi dan melaporkan *syntax error* ketika token tidak sesuai grammar.
- Membangun representasi sintaktis program yang siap dikonsumsi tahap berikutnya.

1.2 Context-Free Grammar (CFG)

Grammar bahasa pemrograman umumnya didefinisikan sebagai *Context-Free Grammar* (CFG).

Definisi. Sebuah CFG adalah 4-tupel $G = (V, \Sigma, R, S)$, di mana: V adalah himpunan non-terminal, Σ adalah himpunan terminal (token), $R \subseteq V \times (V \cup \Sigma)^*$ adalah himpunan production rules, dan $S \in V$ adalah simbol awal.

Dalam konteks Arion, Σ adalah himpunan token yang diemisikan oleh lexer pada Milestone 1, V adalah himpunan non-terminal seperti `<program>`, `<statement>`, dan `<expression>`, sedangkan $S = \text{<program>}$.

1.3 Parse Tree dan AST

Parse tree adalah representasi lengkap dari proses derivasi suatu string oleh CFG. Setiap node internal pada parse tree adalah non-terminal, sedangkan setiap daun adalah terminal. Berbeda dengan *Abstract Syntax Tree* (AST), parse tree memuat seluruh simbol gramatikal termasuk separator, parentheses, dan non-terminal antara, sedangkan AST hanya menyimpan informasi struktural yang relevan secara semantis. Pada Milestone 2 ini, keluaran yang diminta adalah **parse tree**, bukan AST.

1.4 Recursive Descent Parsing

Recursive descent parsing adalah teknik parsing top-down di mana setiap non-terminal $A \in V$ direpresentasikan oleh satu fungsi rekursif. Tubuh fungsi `parseA()` mengikuti aturan produksi $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots$ secara langsung: percabangan grammar diterjemahkan menjadi percabangan kondisi berbasis token *lookahead*, sedangkan rekursi grammar diterjemahkan menjadi rekursi pemanggilan fungsi.

Dengan *single-token lookahead* (LL(1)) yang ditingkatkan dengan beberapa *multi-token lookahead* secara terbatas, parser dapat memilih cabang produksi yang tepat tanpa *backtracking*. Sifat ini menjamin kompleksitas waktu yang linier $O(n)$ terhadap panjang barisan token.

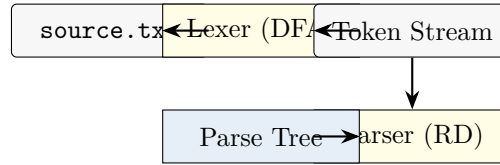


Figure 1: Pipeline kompilasi Arion: keluaran lexer (Milestone 1) menjadi masukan parser (Milestone 2).

2 Design and Implementation

2.1 Overview

Komponen parser diimplementasikan dalam C++17 sebagai kelanjutan langsung dari program Milestone 1. Antarmuka antara lexer dan parser adalah `std::vector<Token>` yang dihasilkan oleh `Lexer::tokenize()`. Source code parser terbagi menjadi dua berkas: `src/header/parser.hpp` mendeklarasikan struct `ParseNode`, kelas `Parser`, serta exception `SyntaxError`; sedangkan `src/lib/parser.cpp` mengimplementasikan seluruh fungsi rekursif serta utilitas pencetakan parse tree (`printTree`, `writeTree`).

Setiap node parse tree direpresentasikan oleh struct sederhana berikut.

```

1 struct ParseNode {
2     std::string label;
3     std::vector<ParseNode*> children;
4 };
  
```

`label` menyimpan nama non-terminal (mis. `<expression>`) atau representasi terminal (mis. `ident(x)`, `becomes`). Penghapusan parse tree dilakukan secara rekursif melalui `destroyParseTree()` untuk menghindari kebocoran memori.

2.2 Hard-coded Grammar

Sesuai spesifikasi, grammar di-*hardcode* di dalam program melalui satu fungsi per non-terminal. Tabel 1 merangkum keseluruhan production rules yang diimplementasikan.

Table 1: Production rules grammar Arion yang di-*hardcode* pada parser.

Non-terminal	Produksi
<code><program></code>	<code><program-header> <declaration-part> <compound-statement> period</code>
<code><program-header></code>	<code>programsy ident semicolon</code>
<code><declaration-part></code>	<code>(<const-declaration>)* (<type-declaration>)* (<var-declaration>)* (<subprogram-declaration>)*</code>
<code><const-declaration></code>	<code>constsy (ident eql <constant> semicolon)+</code>
<code><constant></code>	<code>charcon string [(plus minus)? (ident intcon realcon)]</code>

Non-terminal	Produksi
<type-declaration>	typesy (ident eql <type> semicolon)+
<var-declaration>	varsy (<identifier-list> colon <type> semicolon)+
<identifier-list>	ident (comma ident)*
<type>	ident <array-type> <range> <enumerated> <record-type>
<array-type>	arraysy lbrack (<range> ident) rbrack ofsy <type>
<range>	<constant> period period <constant>
<enumerated>	lparent ident (comma ident)* rparent
<record-type>	recordsy <field-list> endsy
<field-list>	<field-part> (semicolon <field-part>)*
<field-part>	<identifier-list> colon <type>
<subprogram-declaration>	<procedure-declaration> <function-declaration>
<procedure-declaration>	proceduresy ident (<formal-parameter-list>)? semicolon <block> semicolon
<function-declaration>	functionsy ident (<formal-parameter-list>)? colon ident semicolon <block> semicolon
<block>	<declaration-part> <compound-statement>
<formal-parameter-list>	lparent <parameter-group> (semicolon <parameter-group>)* rparent
<parameter-group>	<identifier-list> colon (ident <array-type>)
<compound-statement>	beginsy <statement-list> endsy
<statement-list>	<statement> (semicolon <statement>)*
<statement>	<assignment-statement> <if-statement> <case-statement> <while-statement> <repeat-statement> <for-statement> <procedure/function-call> ϵ
<assignment-statement>	<variable> becomes <expression>
<variable>	ident ((lbrack <index-list> rbrack) (period ident))*
<if-statement>	ifsy <expression> thensy <statement> (elsesy <statement>)?
<case-statement>	casesy <expression> ofsy <case-block> (semicolon <case-block>)* endsy
<case-block>	<constant> (comma <constant>)* colon <statement>
<while-statement>	whilesy <expression> dosy <statement>
<repeat-statement>	repeatsy <statement-list> untilsy <expression>
<for-statement>	forsy ident becomes <expression> (tosy downtosy) <expression> dosy <statement>
<procedure/function-call>	ident (lparent <parameter-list> rparent)?
<parameter-list>	<expression> (comma <expression>)* ϵ
<expression>	<simple-expression> (<relational-operator> <simple-expression>)?
<simple-expression>	(plus minus)? <term> (<additive-operator> <term>)*
<term>	<factor> (<multiplicative-operator> <factor>)*
<factor>	intcon realcon charcon string (lparent <expression> rparent) (notsy <factor>) <procedure/function-call> <variable>
<relational-operator>	eql neq gtr geq lss leq
<additive-operator>	plus minus orsy
<multiplicative-operator>	times rdiv idiv imod andsy

2.3 Recursive Descent Architecture

Setiap non-terminal pada Tabel 1 dipetakan ke satu method private `parseX()` pada kelas `Parser`. Pemanggilan dimulai dari `parseProgram()` dan terus turun secara rekursif sesuai struktur grammar. Diagram 2 menunjukkan hierarki pemanggilan utama.

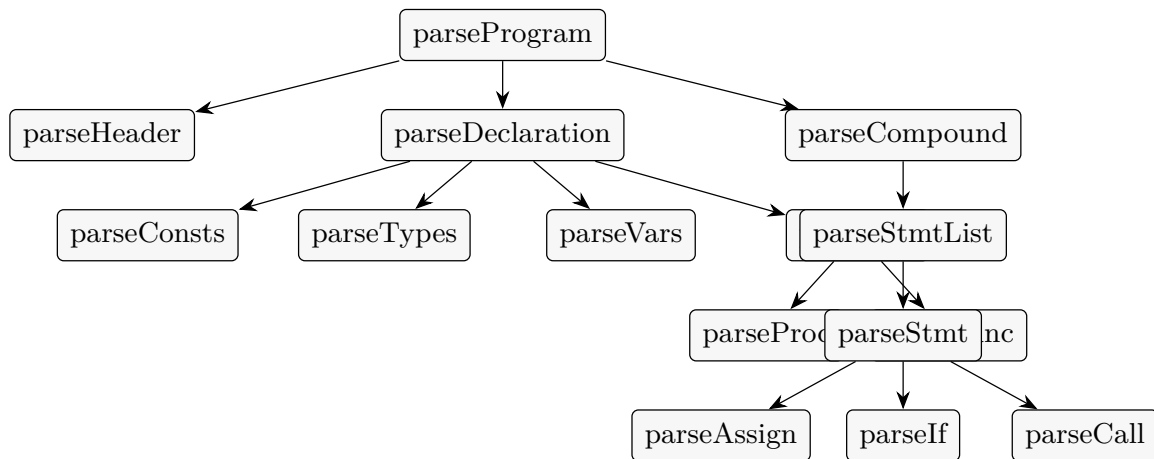


Figure 2: Hierarki pemanggilan utama recursive descent parser. Setiap node menggambarkan satu fungsi anggota kelas **Parser**; subtree ekspresi (**parseExpr** → **parseSimple** → **parseTerm** → **parseFactor**) tidak ditampilkan karena dipanggil dari berbagai cabang.

2.4 Parsing Expression: Operator Precedence

Hierarki ekspresi mengikuti tingkatan presedensi standar dengan empat lapis non-terminal. **<expression>** menangani operator relasional (presedensi terendah), **<simple-expression>** menangani operator aditif termasuk unary **+/-**, **<term>** menangani operator multiplikatif, dan **<factor>** menangani unit terkecil (literal, variabel, pemanggilan, atau ekspresi berkurung). Pendekatan ini menghasilkan parse tree dengan struktur berdasarkan presedensi tanpa memerlukan tabel presedensi eksplisit.

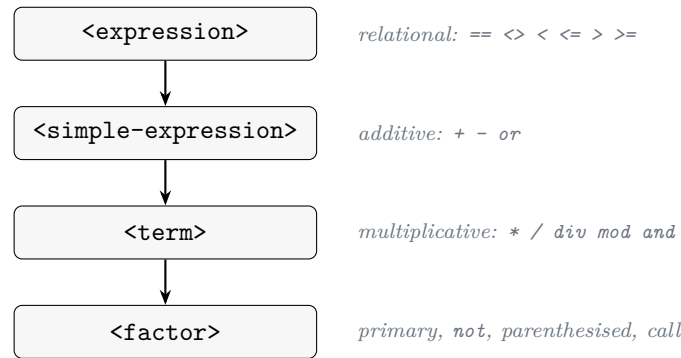


Figure 3: Stratifikasi ekspresi: presedensi naik dari atas ke bawah.

2.5 Disambiguasi: Assignment vs Procedure Call

Salah satu titik ambigu dalam grammar adalah **<statement>**: ketika token saat ini adalah **ident**, statement dapat berupa **<assignment-statement>** (mis. **p.x := 3**) maupun **<procedure/function-call>** (mis. **writeln('hi')**). Karena **<variable>** dapat memuat indeks array dan akses field sembarang panjang sebelum mencapai **becomes**, satu token lookahead tidak cukup.

Permasalahan ini diselesaikan oleh utilitas **isAssignFollowed()**: dari posisi token saat ini, fungsi men-skip seluruh [...] dan **.ident** lookahead-only (tanpa mengkonsumsi token), kemudian memeriksa apakah token berikutnya adalah **becomes**. Bila ya, parser memilih jalur **parseAssign()**; bila tidak, parser memilih **parseCall()**. Dengan demikian, parser tetap deterministik tanpa backtracking, sebab keputusan percabangan dilakukan murni melalui pemeriksaan tabel token in-place.

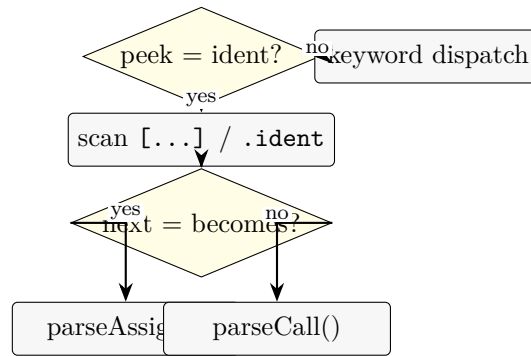


Figure 4: Disambiguasi statement berbasis multi-token lookahead non-konsumtif.

2.6 Error Handling

Setiap pelanggaran grammar dilaporkan melalui exception **SyntaxError** dengan pesan yang menyertakan nomor baris token, lexeme yang dilihat, dan ekspektasi parser. Tiga utilitas internal mendukung mekanisme ini.

- **expect(t)** mengkonsumsi token jika tipe-nya cocok, atau melempar **SyntaxError** jika tidak.
- **checkNotUnknown(t)** memvalidasi bahwa token bukan **UNKNOWN**; token **UNKNOWN** dari lexer langsung dilaporkan sebagai syntax error sehingga error leksikal terpropagasi rapi.
- Pesan error mengikuti format "Syntax error at line X: unexpected token T, expected E".

Setelah **parseProgram()** kembali, parser mengecek bahwa token tersisa adalah **EOF**; jika tidak, kelebihan token dianggap sebagai error.

2.7 Parse Tree Output

Parse tree dicetak dalam representasi pohon ASCII menggunakan karakter box-drawing Unicode (*tee*, *elbow*, dan *vertical bar*) untuk menggambar cabang. Implementasi rekursif pada **printTreeImpl()** mempertahankan prefix dinamis untuk setiap level, serta membedakan child terakhir dari child antara. Output yang sama ditulis ke file via **writeTree()** sehingga setiap pengujian menghasilkan pasangan **output-{n}.txt** yang dapat diverifikasi.

2.8 Tokenization Flow

Diagram 5 memberikan gambaran umum eksekusi **Parser::parse()** pada level statement. Whitespace dan komentar telah dibuang oleh lexer, sehingga parser hanya bekerja pada barisan token.

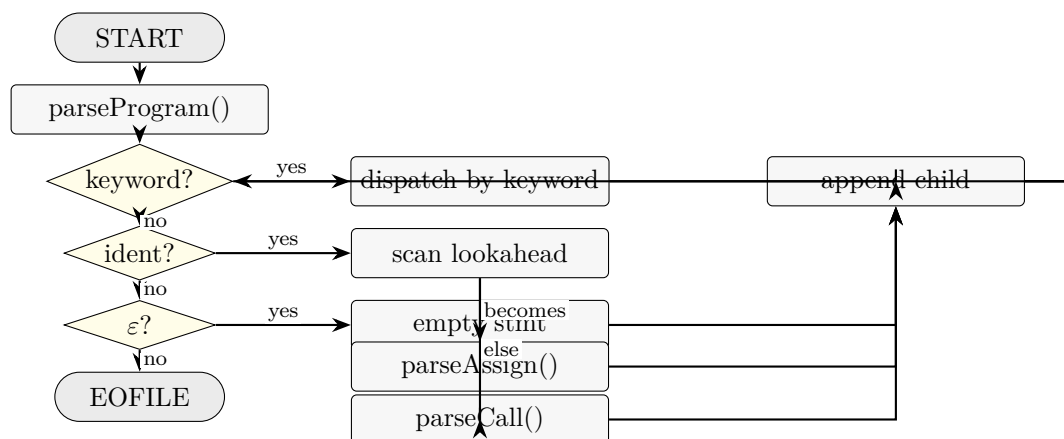


Figure 5: Alur eksekusi pada level statement di dalam recursive descent parser.

2.9 Contoh Parse Tree

Sebagai ilustrasi, perhatikan ekspresi $a + b * 2$. Setelah dipanggil melalui `parseExpr()`, parser membangun pohon berikut yang merefleksikan presedensi $*$ di atas $+$.

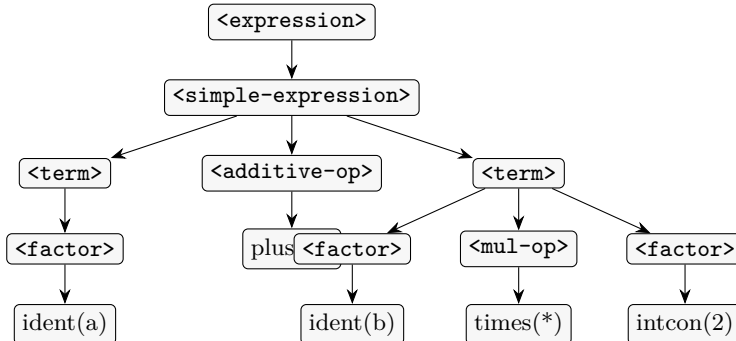


Figure 6: Parse tree untuk ekspresi $a + b * 2$.

3 Testing

Pengujian (*testing*) program dilakukan menggunakan testcase yang sudah dibuat dan disimpan pada direktori `test/milestone-2/`. Setiap testcase memiliki pasangan berkas input-output dengan format nama `input-{n}.txt` dan `output-{n}.txt`. Tujuh kasus disusun untuk meliputi kasus golden-path, fitur tipe data lanjutan, kontrol alur, prosedur/fungsi, hingga skenario error.

3.1 Lokasi File Pengujian

- Berkas input: `test/milestone-2/input-1.txt` s.d. `test/milestone-2/input-7.txt`
- Berkas output: `test/milestone-2/output-1.txt` s.d. `test/milestone-2/output-7.txt`

3.2 Ringkasan Kasus Uji

No	Fokus	Deskripsi singkat
1	Golden path	Program minimal: header, <code>var</code> , assignment, dan pemanggilan <code>writeln</code> .
2	Tipe data lanjutan	Deklarasi <code>const</code> , <code>type</code> (range, enumerated, array bertingkat), dan akses array dua dimensi.
3	Subprogram	Prosedur tanpa parameter, prosedur dengan parameter, dan fungsi yang mengembalikan nilai.
4	Kontrol alur	<code>if-else</code> , <code>case</code> , <code>while</code> , <code>repeat-until</code> , <code>for-to</code> , dan <code>for-downto</code> .
5	<code>record</code> & field	Deklarasi <code>record</code> , akses field via <code>.</code> , akses array dengan indeks variabel.
6	Error: urutan deklarasi	<code>type</code> dideklarasikan setelah <code>var</code> (melanggar urutan strict).
7	Error: token leksikal	Konstanta menggunakan <code>=</code> alih-alih <code>==</code> ; token <code>unknown</code> terpropagasi ke parser.

Table 2: Ringkasan tujuh kasus uji Milestone 2.

3.3 Hasil Uji Berdasarkan Screenshot

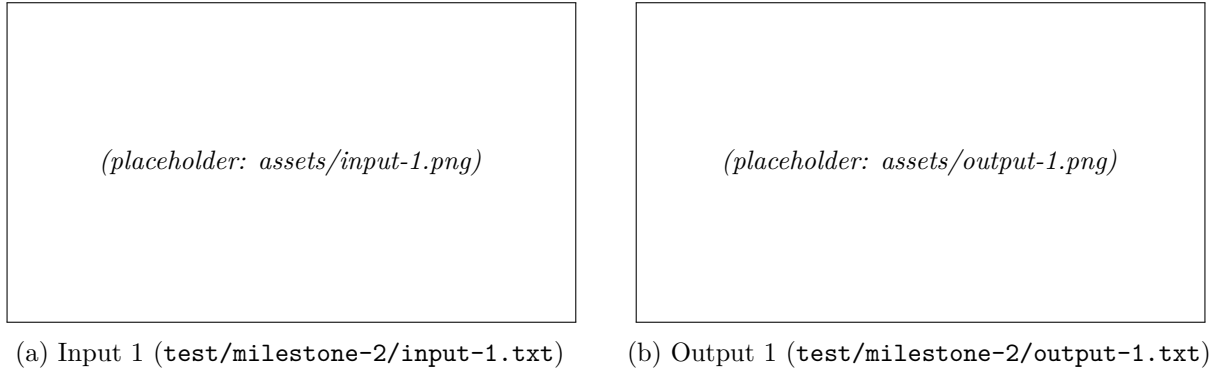


Figure 7: Pasangan hasil uji kasus 1 (golden-path program minimal).

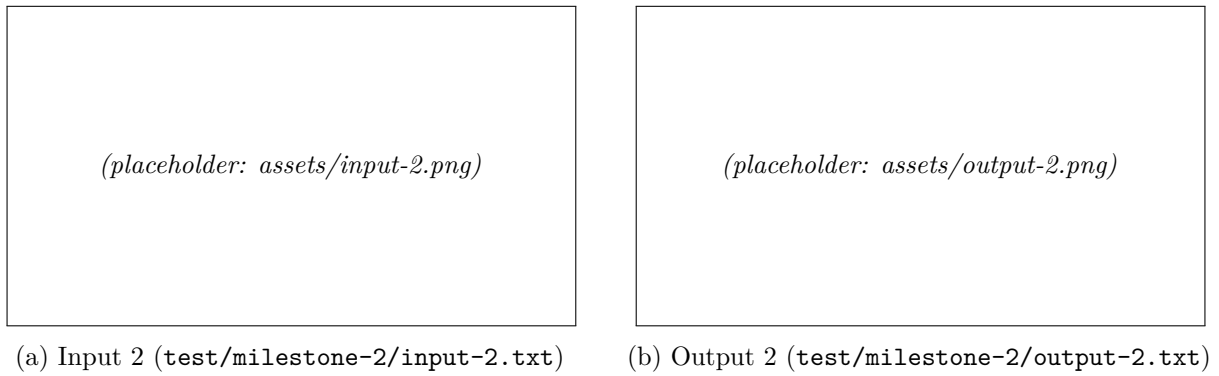


Figure 8: Pasangan hasil uji kasus 2 (deklarasi `const`, `type`, array bertingkat).

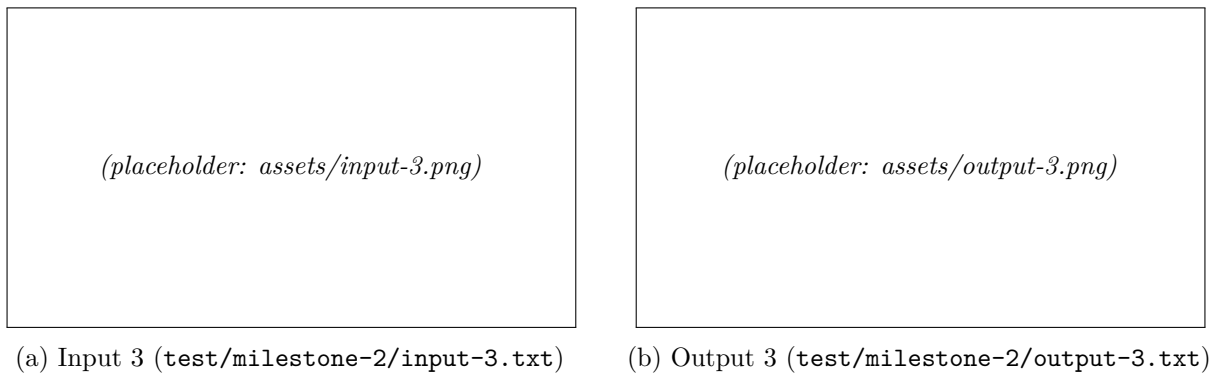
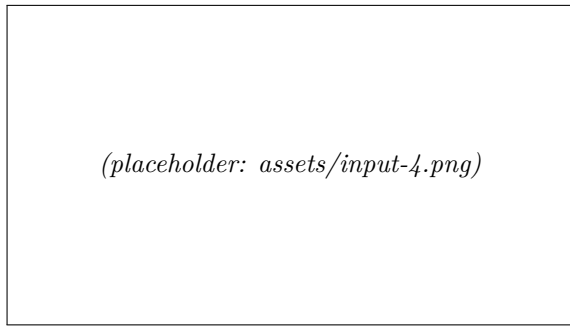
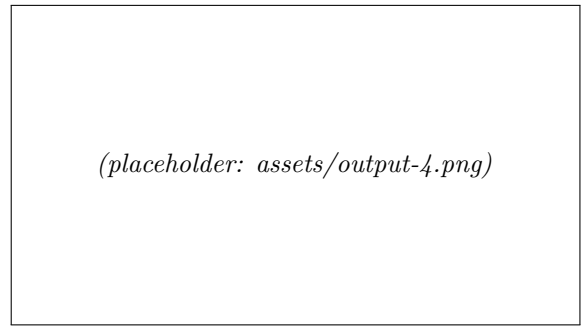


Figure 9: Pasangan hasil uji kasus 3 (prosedur dan fungsi).

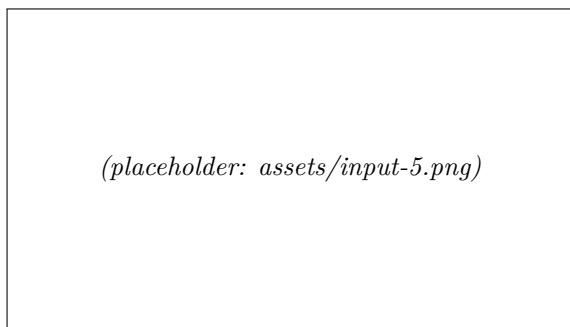


(a) Input 4 (`test/milestone-2/input-4.txt`)

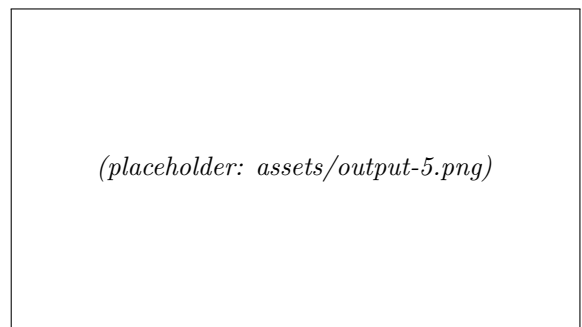


(b) Output 4 (`test/milestone-2/output-4.txt`)

Figure 10: Pasangan hasil uji kasus 4 (seluruh statement kontrol alur).

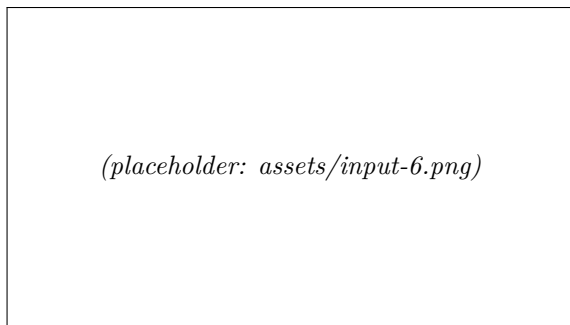


(a) Input 5 (`test/milestone-2/input-5.txt`)

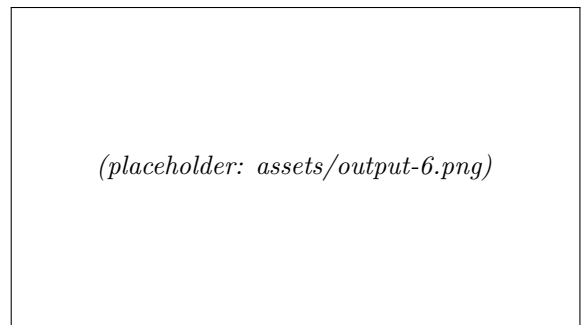


(b) Output 5 (`test/milestone-2/output-5.txt`)

Figure 11: Pasangan hasil uji kasus 5 (**record** dan akses field).



(a) Input 6 (`test/milestone-2/input-6.txt`)



(b) Output 6 (`test/milestone-2/output-6.txt`)

Figure 12: Pasangan hasil uji kasus 6 (error: **type** dideklarasikan setelah **var**).

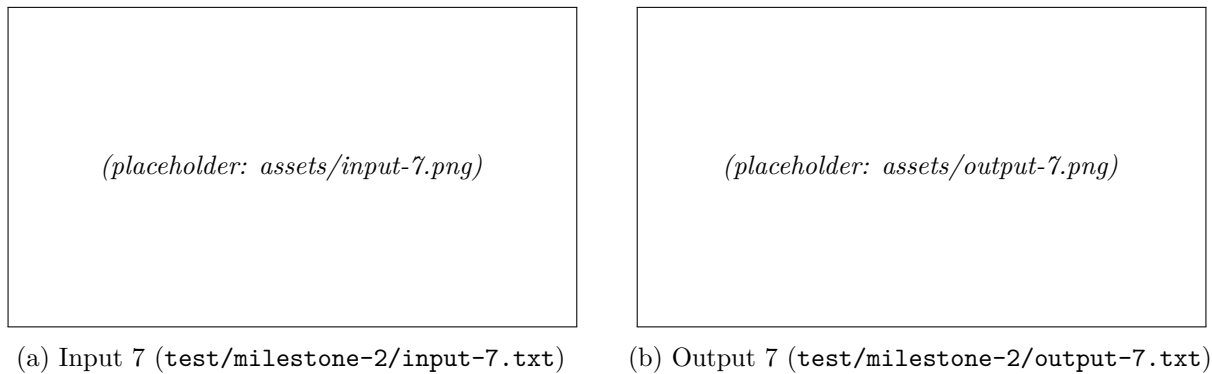


Figure 13: Pasangan hasil uji kasus 7 (error: = alih-alih == pada deklarasi konstanta).

4 Conclusion

Dari pengerjaan Milestone 2 ini, komponen *Syntax Analyzer* (Parser) untuk Arion Compiler berhasil diimplementasikan dan diuji sebagai kelanjutan langsung dari Lexer Milestone 1. Pembuatan parser ini sepenuhnya menggunakan bahasa pemrograman C++17, dengan menerapkan pendekatan *Recursive Descent* LL(1) yang ditingkatkan oleh *multi-token lookahead* terbatas pada titik ambigu tertentu. Secara keseluruhan, program parser sudah sanggup berjalan dengan baik sesuai spesifikasi grammar bahasa Arion yang diharapkan.

Salah satu poin penting dari arsitektur parser ini adalah pemetaan langsung satu non-terminal ke satu fungsi rekursif. Pendekatan ini menjadikan grammar yang ada di laporan dapat dilacak secara linier ke implementasi C++, tanpa backtracking dan tanpa parser generator eksternal. Hal ini berdampak baik pada kinerjanya yang efisien dengan kompleksitas waktu yang linier, yakni $O(n)$ terhadap panjang barisan token.

Program parser ini sukses membangun *parse tree* untuk seluruh konstruksi bahasa Arion yang terdaftar pada spesifikasi: deklarasi **const**, **type**, **var**, prosedur dan fungsi (dengan dan tanpa parameter), seluruh statement kontrol alur (**if**, **case**, **while**, **repeat**, **for**), tipe data lanjutan (**array**, **record**, **range**, **enumerated**), serta ekspresi dengan presedensi yang benar. Selain itu, parser terbukti mampu mendeteksi dan melaporkan kesalahan sintaks dengan pesan informatif yang menyertakan nomor baris dan ekspektasi token.

Kesimpulannya, keluaran langsung dari pengerjaan sintaktis ini terbukti sukses mencetak *parse tree* yang rapi, utuh, dan terstruktur. Karena keluaran parse tree ini sudah merepresentasikan struktur hierarkis program secara penuh, ini bakal mempermudah kita mengerjakan langkah lanjut di tahapan Milestone berikutnya. Intinya, landasan parse tree ini sudah siap digunakan untuk pekerjaan membangun representasi *Abstract Syntax Tree* (AST) maupun *Symbol Table* pada pengerjaan *Semantic Analysis* selanjutnya.

4.1 Saran

Berdasarkan hasil pengerjaan Milestone 2 ini, beberapa saran untuk pengembangan ke depan adalah sebagai berikut.

1. **Error recovery yang lebih baik.** Saat ini, parser menggunakan strategi *panic-mode* sederhana: satu syntax error akan menghentikan proses parsing. Ke depannya, implementasi *phrase-level recovery* atau *synchronizing tokens* (mis. **semicolon**, **end**) akan memungkinkan parser melanjutkan proses dan melaporkan beberapa error dalam satu eksekusi.
2. **Reduksi parse tree menjadi AST.** Parse tree saat ini memuat seluruh non-terminal antara seperti `<simple-expression>` dan `<term>` yang hanya berfungsi untuk mengkodekan presedensi. Pada Milestone berikutnya, pohon dapat diciutkan menjadi AST yang lebih ringkas untuk mempermudah analisis semantis.
3. **Integrasi dengan tahap berikutnya.** Parser telah dirancang secara modular sehingga *parse tree* yang dihasilkan siap dikonsumsi oleh *semantic analyzer* pada milestone selanjutnya. Disarankan agar antarmuka

antara Parser dan Semantic Analyzer dipastikan konsisten sejak dini, terutama mengenai representasi simbol dan tipe.

References

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
- [2] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Pearson, 2006.
- [3] “Compiler Design - Syntax Analysis,” TutorialsPoint. https://www.tutorialspoint.com/compiler_design/compiler_design_syntax_analysis.htm
- [4] R. Spivak, “Let’s Build A Simple Interpreter, Part 7: Abstract Syntax Trees,” <https://ruslanspivak.com/lsbasi-part7/>
- [5] “An Introduction to Makefiles,” GNU Make Manual. https://www.gnu.org/software/make/manual/html_node/Introduction.html

Appendix

GitHub Repository

<https://github.com/jsndwrld/DCL-Tubes-IF2224-2026>

Task Distribution

NIM	Nama	Kontribusi
13524026	Made Branenda Jordhy	25%
13524028	Muhammad Nur Majiid	25%
13524034	Jason Edward Salim	25%
13524068	Athilla Zaidan Zidna Fann	25%